SIGNALDESK

Incident management, from local app to GitOps

Original React + FastAPI + PostgreSQL application with a tested delivery path through CI, containers, Terraform, Kubernetes, observability, and ArgoCD.

PREPARED CAPSTONE

LIVE EKS DEPLOYMENT

An incident workflow that closes the loop

Each record carries service, severity, status, owner, description, and timestamps.

OPEN

Declare and assign

INVESTIGATING

Investigate impact

MITIGATED

Restore service

RESOLVED

Record closure

Dashboard

Status totals, recent incidents, search, filters, and a responsive detail drawer.

Recovery loop

Resolved incidents can reopen; invalid transitions return HTTP 409.

Three layers, one incident record

Local Compose and the EKS application are running with published SHA-tagged images.

Browser and UI

React + Vite dashboard
Responsive CSS
Nginx runtime
/api routed to FastAPI



API and workflow

FastAPI REST endpoints
SQLAlchemy ORM
Readiness and /metrics
Alembic migrations



Persistent data

PostgreSQL incidents table
Create/update timestamps
Resolved timestamp
PVC in Kubernetes

Local: docker compose up --build | Cluster: NLB -> ingress-nginx -> frontend / backend -> PostgreSQL

The development gates have real local results

These checks verify code and containers; they do not prove a GitHub or EKS run.

API tests

8 isolated pytest cases across health, incident CRUD, workflow rules, and metrics.

Browser tests

4 Chrome checks at desktop and mobile widths, including API failure states.

Compose

Frontend, backend, and PostgreSQL ran together; app images ran as non-root.

The same API passed live browser workflows through the public NLB; Prometheus now scrapes both backend replicas.

Quality gates before image publication

A push to main triggered a verified run; tests, builds, and both Trivy scans passed before GHCR publication.

01 VERIFY

pytest + Vite build

02 BUILD

Backend + frontend images

03 SCAN

Trivy HIGH / CRITICAL

04 PUBLISH

GHCR full SHA tags

Run #1 (33310e2) passed; both scans found 0 HIGH/CRITICAL findings at that time. Both public GHCR images carry the matching full SHA.

Terraform provisioned the AWS foundation

Approved live apply completed in Sydney; a later refresh-based plan reported no changes.

Network

VPC
2 public subnets
2 private subnets
Internet gateway + NAT

Compute

EKS cluster
Managed worker node group
2 desired m7i-flex.large nodes
Pod Identity Agent

Storage

EBS CSI add-on
gp3 StorageClass
PostgreSQL PVC
Prometheus and Grafana PVCs

LIVE: 2 READY NODES, EKS + EBS + VPC | CONSOLE CAPTURES AND TEARDOWN: PENDING

Replicas, routing, and one migration owner

Two frontend and two backend Pods are Ready; PostgreSQL has a bound encrypted gp3 claim.

Frontend

2 replicas
Nginx container
ClusterIP Service
Serves the React bundle

Backend

2 replicas minimum
CPU HPA up to 5
ClusterIP Service
/ready checks PostgreSQL

PostgreSQL

Single StatefulSet
Persistent gp3 claim
Connection in Secret
Back up before teardown

A pre-install / pre-upgrade Job runs Alembic once before the backend rolls. Replicas do not race to migrate the schema.

One internet-facing NLB, two app paths

Controller chart 1.14.0 and matching v2.14.0 IAM policy created the public NLB.

AWS Load Balancer

NLB Service class
Internet-facing scheme
IP targets in the VPC



ingress-nginx

Receives public HTTP
Routes on path
Service gets NLB hostname



Application

/ -> React frontend
/api -> FastAPI backend
Both Services are ClusterIP

Desktop and mobile workflows passed through the NLB. Demo route is HTTP without app authentication; use test data only.

From /metrics to a live Grafana panel

Prometheus targets and Grafana dashboard were verified through local port-forwards.

FastAPI

Request, latency, creation, transition counters

ServiceMonitor

Scrape backend /metrics every 30s

Prometheus

Discover the app across namespaces

Grafana

Request rate and p95 latency panels

Both backend targets are UP; the Grafana screenshot shows request-rate and p95 latency data after live traffic.

Helm demo, then ArgoCD ownership

The manual release was removed before ArgoCD took ownership of the application.

1 Manual Helm

Install the app with a published SHA. Capture helm list and healthy Pods.

2 Handoff

Uninstall only the app release. Keep DB, Secrets, and monitoring.

3 ArgoCD

Sync chart from Git. PreSync Job migrates; tracked SHA values roll Pods.

ArgoCD is Synced/Healthy. Commit 3a780f9 updated the tracked pointer; both Deployments run published image SHA 49e3313.

Claims follow their evidence

Local, GitHub, public browser, and cluster command results are captured.

Locally verified

8 pytest cases and 4 Chrome checks
Three-service Compose stack
Non-root application images
Terraform fmt/validate/offline plan
Helm render and schema checks

Live GitHub and AWS

Actions published both SHA images
Two EKS worker nodes Ready
Public browser workflow passed
Prometheus targets UP
ArgoCD Synced / Healthy

Pending: video, AWS Console captures, and eventual teardown proof.

The mandatory video starts with these slides

Then show the complete live path, with actual evidence on screen.

Video order

1. Present slide deck first
2. Incident workflow and tests
3. New commit + push, run, GHCR
4. Terraform and AWS Console
5. Helm, Ingress, monitoring, ArgoCD

Evidence to preserve

Commit history and test output
Running Pods, Services, Helm list
Public browser route
Prometheus target + Grafana panel
ArgoCD update and teardown

The deployment is live; the mandatory recording remains pending. The rubric awards zero for this module without it.

A traceable path needs clear gates and owners

GitHub and GHCR are verified; the approved AWS deployment is running.

Quality before release

Tests and frontend build gate image creation. Trivy gates both image pushes.

One owner per resource

Terraform owns AWS; Helm bootstraps controllers; ArgoCD owns the app after handoff.

Operations matter

Guard cluster context, keep Secrets external, migrate once, monitor live, and destroy paid resources.

Remaining: video, AWS Console captures, backup, and paid-resource teardown after grading.